# CSE 242 — Assignment 3
# SVMs, K-means, and K-NN

## Question 1: Hard-margin linear SVM

### (a) Plot and support vectors

Figure 1 shows the six training points. The support vectors are points **1**, **5**, and **6**: point 1 is the nearest positive point to the negative class, while points 5 and 6 are the nearest negative points to the positive class. They lie on the two closest parallel supporting lines, $x_1 + x_2 = 2$ and $x_1 + x_2 = 1$, respectively. The other three points are farther from the separating line.

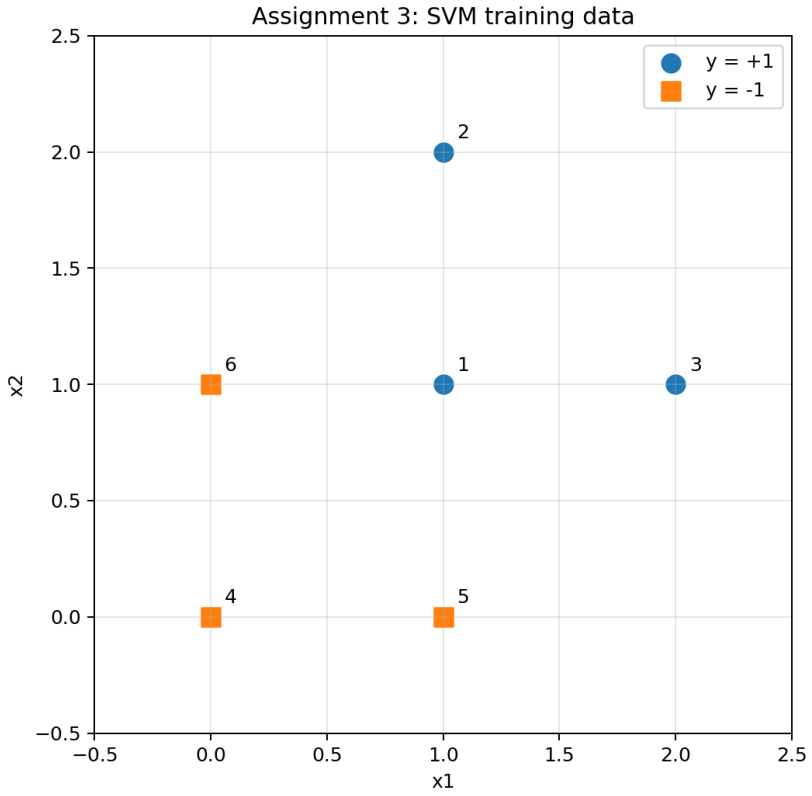



Figure 1: Training data. Point numbers match the assignment table.

### (b) Boundary and geometric margin

For the canonical hard-margin SVM, support vectors satisfy $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Applying this to the three support vectors gives

$$\begin{aligned} w_1 + w_2 + b &= 1 && \text{(point 1)}, \\ w_1 + b &= -1 && \text{(point 5)}, \\ w_2 + b &= -1 && \text{(point 6)}. \end{aligned}$$

The last two equations imply $w_1 = w_2$. Substituting into the first gives $\mathbf{w} = (2, 2)$ and $b = -3$. Thus the maximum-margin decision boundary is

$$\boxed{2x_1 + 2x_2 - 3 = 0}, \qquad \text{or equivalently} \qquad x_1 + x_2 = 1.5.$$

The geometric margin, measured from the boundary to the closest training point, is

$$\boxed{\gamma = \frac{1}{\|\mathbf{w}\|_2} = \frac{1}{\sqrt{2^2 + 2^2}} = \frac{1}{2\sqrt{2}} \approx 0.35355.}$$

For reference, the full width between the two supporting lines is $2\gamma = 1/\sqrt{2} \approx 0.70711$.

## Question 2: K-means

### (a) Update rules

For each point, I assign it to the centroid with the smallest squared Euclidean distance. Squaring does not change which centroid is closest. If distances tie, `argmin` picks the lowest cluster index. I then replace each nonempty cluster's centroid with the mean of its assigned points. An empty cluster keeps its previous centroid. The centroid update returns a copy, so it does not alter the input array in place.

### (b) Results for different values of $k$

The dataset has 4,000 points generated from four groups with standard deviation 0.60. I used seed 0 and ran 100 assignment/update iterations for each $k$. The generating labels were not used by K-means.

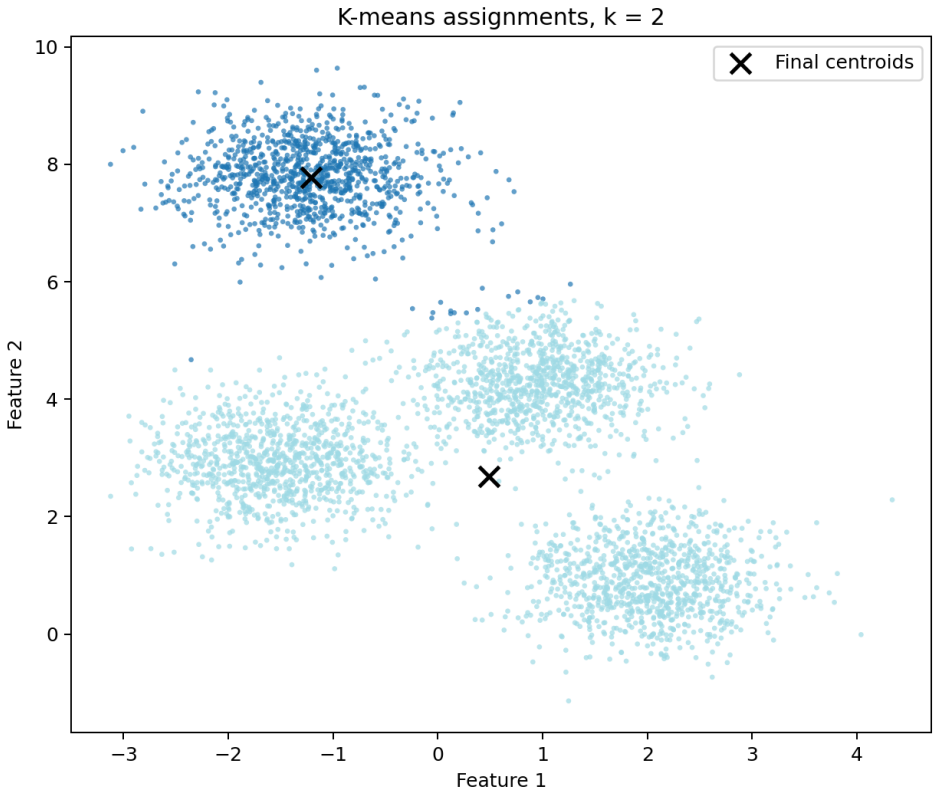



Figure 2: K-means assignments and final centroids for $k = 2$.

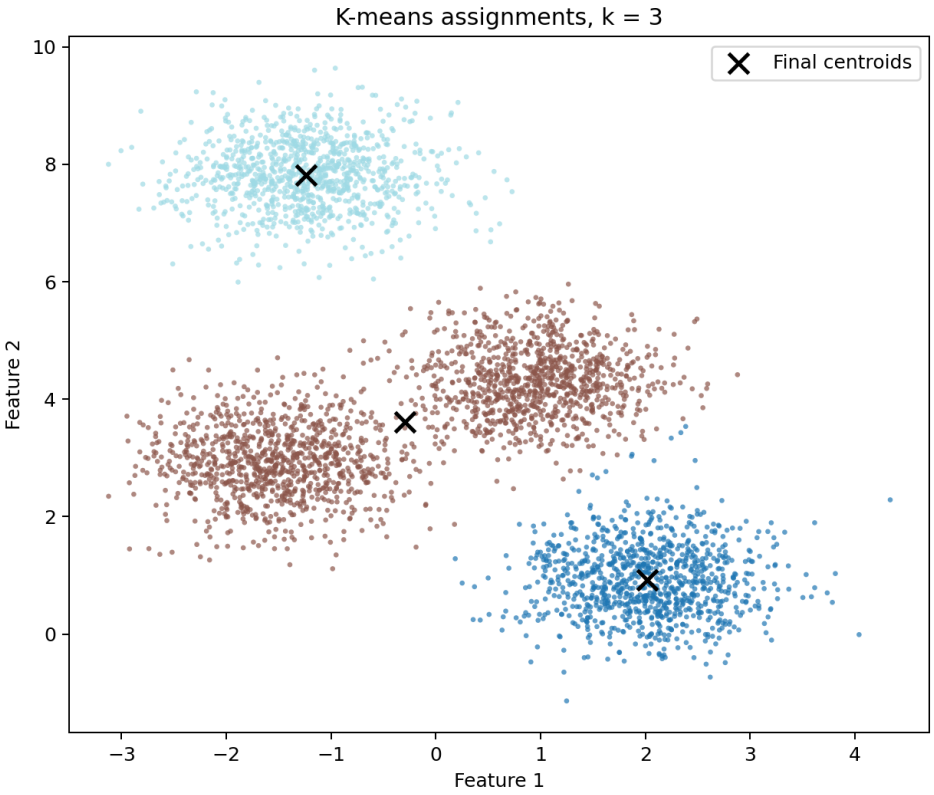



Figure 3: K-means assignments and final centroids for $k = 3$.

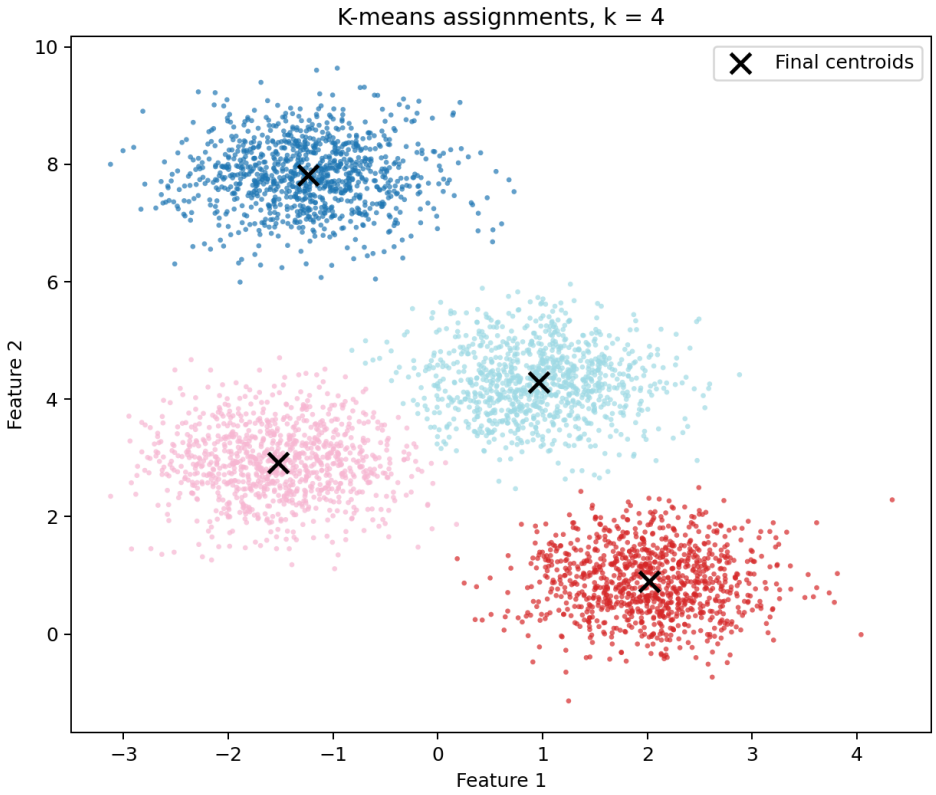



Figure 4: K-means assignments and final centroids for $k = 4$.

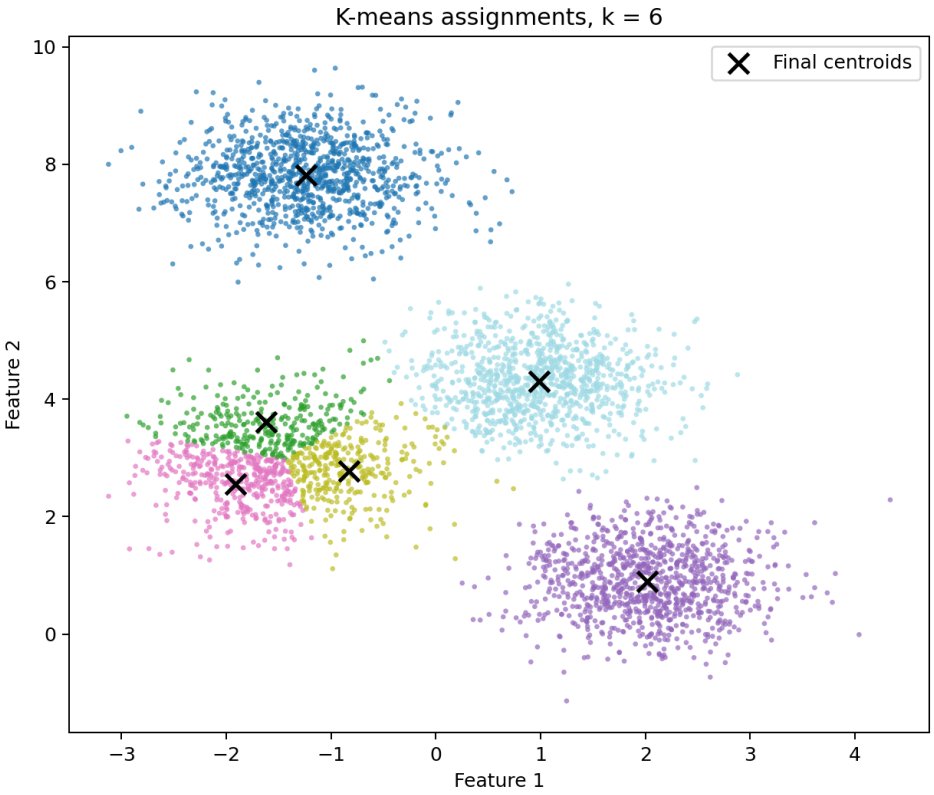



Figure 5: K-means assignments and final centroids for $k = 6$.

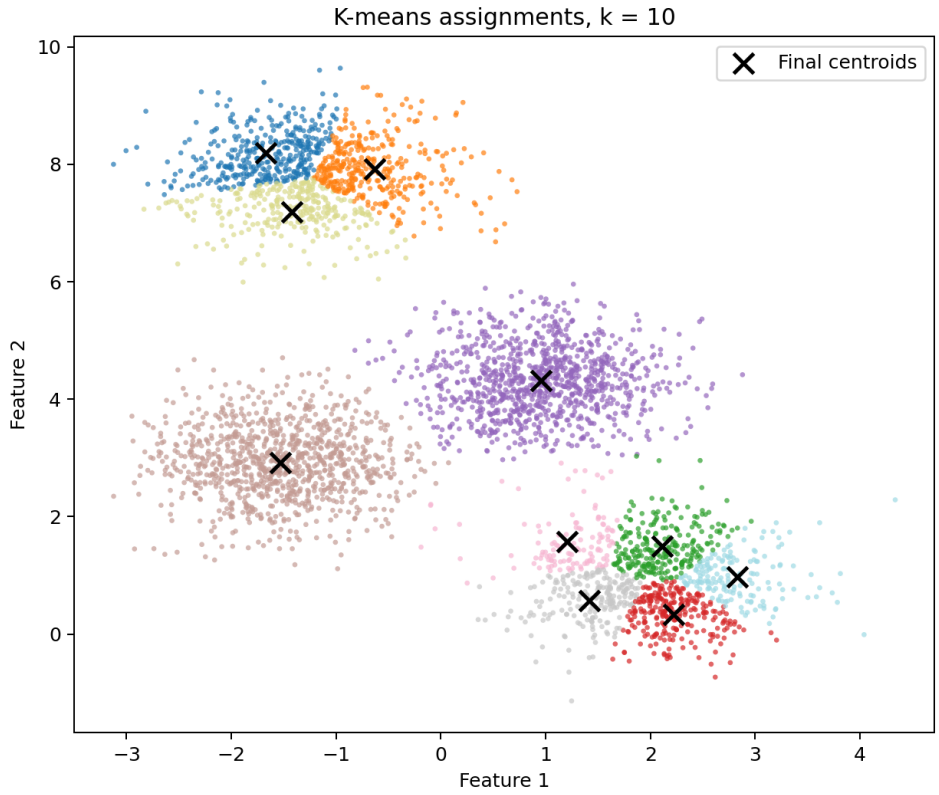



Figure 6: K-means assignments and final centroids for $k = 10$.

With $k = 2$ or 3, some of the four visible generating groups must share a cluster. At $k = 4$, the result separates the four visible groups. Increasing $k$ to 6 and 10 subdivides some groups; those extra clusters do not represent additional groups in the data generation process.

## Question 3: Noisy-label SVM and K-NN repair

I generated independent training and test sets, each with 1,000 positive points from $\mathcal{N}((1, 1), I)$ and 1,000 negative points from $\mathcal{N}((-1, -1), 3I)$. The test labels remained clean. The features and test set stayed fixed across 20 trials with seed 0. In each trial, an original positive training label flipped with probability 0.35 and an original negative label with probability 0.20.

The baseline trained an RBF SVM with $C = 1$ and $\gamma = 0.01$ on the noisy labels. For the repair method, each training point received the majority label of its five nearest *other* training points. All votes used the same original noisy-label array. If neighbor distances tied, the lower training-row index won. A fresh SVM with the same settings was trained on the repaired labels. Both methods used the same corruption in each trial.

| Method | Mean clean-test accuracy (20 trials) |
|---|---|
| Noisy-label SVM | 0.8280 (82.80%) |
| K-NN repair, then SVM | 0.8644 (86.44%) |
| Difference | +0.0364 (+3.64 percentage points) |

Table 1: Accuracy on the clean test set. Per-trial values are in `results/noise_comparison.csv`.

K-NN repair helped in this run. Nearby examples often share a true class, so voting can remove isolated label flips before the SVM is trained. It can also hurt near overlapping classes or where one class is locally outnumbered. The observed improvement is therefore a result of this seeded experiment, not a guarantee for every dataset.

**Outside help.** OpenAI Codex assisted with code implementation and validation.